

1. DÉFINITIONS FONDAMENTALES

Objet / instance : entité identifiable avec un **état** (attributs), un **comportement** (méthodes) et une **identité**.

Classe : modèle/type décrivant une structure (attributs) et des operations communes. **Instanciation** : création d'un objet a partir de la classe, généralement avec new.

Abstraction : ne garder que les caractéristiques utiles d'un concept. **Encapsulation** : cacher l'état et l'implémentation internes et exposer une interface de services.

Modularite : découper en modules faiblement couplés. **Hierarchie** : généralisation/spécialisation selon la relation "est une sorte de".

Polymorphisme : une meme abstraction peut avoir plusieurs formes/implémentations. **Persistence** : conserver l'état au-dela de l'exécution (fichier, base de données, serialisation).

Type statique : type de la variable, contrôle les appels autorisés a la compilation. **Type dynamique** : classe reelle de l'objet, utilisée pour les méthodes redéfinies.

2. CLASSE, OBJET, CONSTRUCTEUR

```
public class Point {
    private int x, y;
    private static int nb;

    public Point(int x, int y) {
        this.x = x; this.y = y; nb++;
    }
    public int getX() { return x; }
    public static int getNb() { return nb; }
}

Point p; // référence seulement
p = new Point(3, 4); // objet créé
int x = p.getX();
```

Mot-cle	À retenir
private	visible dans la classe
(default)	visible dans le package
protected	package + sous-classes selon les regles Java
public	visible partout si la classe est accessible
static	membre de classe, partagé
final	pas de réaffectation; méthode non redéfinissable; classe non extensible

Constructeur : même nom que la classe, aucun type de retour, appelé après new, surcharge possible. Si aucun constructeur n'est declare, Java fournit un constructeur sans argument; des qu'on en declare un, il faut gerer les signatures nécessaires.

this désigne l'objet courant. Une méthode static n'a pas de this.

Passage des arguments : toujours **par valeur**. Pour un objet, la méthode reçoit une copie de la référence : elle peut modifier l'objet pointé, mais réaffecter le parametre ne change pas la reference de l'appelant.

3. SURCHARGE, HÉRITAGE, POLYMORPHISME

Surcharge : même nom, paramètres différents -> choix essentiellement a la compilation. **Redéfinition** : même signature dans une sous-classe -> choix dynamique selon l'objet reel.

```
class Chien extends Animal {
    public Chien(String n) { super(n); }
    @Override public void parler() {
        super.parler();
        System.out.println("Ouaf");
    }
}
```

Animal a = new Chien("Rex"); // upcast implicite
if (a instanceof Chien) {
 Chien c = (Chien) a; // downcast explicite
}

Héritage = "est un". Une classe n'étend qu'une classe; sans extends, elle derive de Object. Les cycles sont interdits. **Composition** = "a un" / "contient un".

Constructeurs : non hérites. super(...) doit appeler un constructeur existant de la superclasse; si elle n'a pas de constructeur sans argument, l'appel explicite est indispensable.

Redéfinition : même signature, visibilité pas plus restrictive; final interdit la redéfinition; les méthodes static sont masquées, pas redéfinies dynamiquement.

4. ABSTRACTION, INTERFACES, GÉNÉRICITÉ

```
abstract class Forme {
    public abstract double aire();
}

interface Affichable {
    void affiche();
    default void debug() { }
    static int id() { return 1; }
}

class Cercle extends Forme
    implements Affichable {
    @Override public double aire() { return 0; }
    @Override public void affiche() { }
}
```

Classe abstraite : non instanciable; peut contenir attributs, constructeurs, méthodes concretes et abstraites. **Interface** : contrat; une classe peut en implémenter plusieurs. Les constantes d'interface sont implicitement public static final.

Conflit de default : si deux interfaces fournissent la meme méthode default, la classe doit la redefinir et peut cibler A.super.f().

Object : méthodes a connaitre

equals() : egalite logique si redefinie; hashCode() : coherent avec equals pour le hachage; toString() : representation textuelle; getClass() : classe reelle.

Généricité

```
class Boite<T> {
    private T valeur;
    public void set(T v) { valeur = v; }
    public T get() { return valeur; }
}
List<String> noms = new ArrayList<>();
```

Généricité : code indépendant d'un type précis tout en gardant le contrôle de types; évite des casts dangereux.

5. LAMBDA ET INTERFACES FONCTIONNELLES

Interface fonctionnelle : interface avec une seule méthode abstraite. Une lambda en fournit l'implémentation concise.

```
@FunctionalInterface
interface Operation {
    long appliquer(int a, int b);
}
Operation add = (a, b) -> a + b;

() -> action()
x -> x * 2
(a,b) -> a + b
(a,b) -> { int r=a+b; return r; }
```

Type	Signature / appel
Function<T,R>	T -> R; apply(x)
Predicate<T>	T -> boolean; test(x)
Supplier<T>	() -> T; get()
Consumer<T>	T -> void; accept(x)

Reference de méthode : Math::pow, System.out::println, String::compareTo, Personne::new. Elle remplace une lambda qui ne fait qu'appeler la méthode compatible.

6. PIÈGES PAGE 1

- **Point p**; ne crée aucun objet; seul **new** alloue l'instance.
- **static** ne peut pas utiliser directement **this** ni un attribut d'instance.
- **final reference** : la référence ne change plus; l'objet n'est pas forcément immuable.
- **Downcast** : peut échouer a l'exécution; verifier le type réel avec **instanceof**.
- **Héritage** uniquement si "est un"; sinon preferer la composition.
- Ne pas confondre **surcharge** (paramètres différents) et **redéfinition** (même signature héritee).
- Une sous-classe concrete doit implémenter toutes les méthodes abstraites requises.
- Une lambda doit etre compatible avec la méthode unique de l'interface fonctionnelle.

7. PACKAGES ET EXCEPTIONS

```
package mon.projet.modèle;
import java.util.ArrayList;
import java.util.*;

if (x < 0)
    throw new IllegalArgumentException("x negatif");

void lire() throws IOException { ... }

try {
    action();
} catch (ExceptionSpecifique e) {
    ...
} finally {
    ...
}

try (PrintWriter w = new PrintWriter(fichier)) {
    w.println("ok");
}
```

Exception : événement anormal qui interrompt le flot normal. **throw** lance une exception; **throws** annonce qu'une méthode peut la propager. **Checked** : traitement ou déclaration obligatoire. **Unchecked** : *RuntimeException / Error*, pas d'obligation de déclaration.

Ordre des catch : du plus spécifique au plus général. **try-with-resources** ferme automatiquement les ressources *AutoCloseable*.

8. COLLECTIONS ET ENTRÉES-SORTIES

Type	Définition / exemples
List	ordonnée, doublons possibles - ArrayList, LinkedList
Set	sans doublons - HashSet, TreeSet
Map	clé -> valeur, clé unique - HashMap, TreeMap

```
Iterator<String> it = liste.iterator();
while (it.hasNext()) {
    String s = it.next();
}

class Etudiant implements Comparable<Etudiant> {
    public int compareTo(Etudiant e) {
        return Integer.compare(note, e.note);
    }
}
Collections.sort(liste);
```

InputStream / OutputStream : octets. **Reader / Writer** : caractères. Cycle : **ouvrir -> lire/écrire -> fermer**. Les flux peuvent être enveloppés : *File -> Buffered -> Data/Object*.

```
BufferedReader in = new BufferedReader(
    new InputStreamReader(System.in));
String ligne = in.readLine();

DataOutputStream out = new DataOutputStream(
    new BufferedOutputStream(
        new FileOutputStream("data.bin")));
```

9. SÉRIALISATION ET JDBC

Sérialisation : objet <-> représentation persistante via *ObjectOutputStream / ObjectInputStream*. La classe implémente *Serializable* (interface marqueur).

```
class MaClasse implements Serializable { }
```

```
try (ObjectOutputStream out = new ObjectOutputStream(
    new FileOutputStream("objet.bin"))) {
    out.writeObject(objet);
}

try (ObjectInputStream in = new ObjectInputStream(
    new FileInputStream("objet.bin"))) {
    MaClasse o = (MaClasse) in.readObject();
}
```

JDBC : *Connection -> Statement/PreparedStatement -> ResultSet -> fermeture*. *PreparedStatement* permet des requetes parametrees avec ? et *setString/setDouble...*

```
try (Connection c = DriverManager.getConnection(url, props);
    PreparedStatement ps = c.prepareStatement(
        "INSERT INTO t(nom) VALUES(?)")) {
    ps.setString(1, nom);
    ps.executeUpdate();
}
```

10. THREADS ET CONCURRENCE

Thread : flot d'exécution dans un processus. **Race condition** : résultat dépendant de l'entrelacement lorsque plusieurs threads modifient une donnée partagée.

```
class T extends Thread {
    public void run() { ... }
}
new T().start();

class Tache implements Runnable {
    public void run() { ... }
}
new Thread(new Tache()).start();
new Thread(() -> action()).start();

t1.join();
Thread.sleep(1000);

start() cree/démarre un nouveau flot puis execute run(). Appeler run() directement = simple appel de méthode. join() attend la fin d'un thread.
synchronized void incrementer() {
    valeur++;
}

synchronized (this) {
    valeur++;
}

private volatile boolean running = true;
```

synchronized : verrou + visibilité + atomicité de la section critique. Méthode d'instance -> verrou de l'objet; méthode *static* **synchronized** -> verrou au niveau de la classe. **volatile** : visibilité entre threads, sans verrou et sans rendre atomique une opération composée.

Piège majeur : *volatile int* **compteur**; ne rend pas **compteur++** atomique.

11. SOCKETS TCP

Socket : extremité d'une communication. IP = machine, port = service. TCP est orienté connexion.

```
// Serveur
ServerSocket serveur = new ServerSocket();
serveur.bind(new InetSocketAddress(5000));
Socket client = serveur.accept(); // bloquant
InputStream in = client.getInputStream();
OutputStream out = client.getOutputStream();
client.close(); serveur.close();

// Client
Socket s = new Socket();
s.connect(new InetSocketAddress("127.0.0.1", 5000));
```

Serveur itératif : traite un client à la fois. **Serveur concurrent** : un thread exécutant par client; le thread principal reste sur *accept()*.

12. PIÈGES À RETENIR ABSOLUMENT

- **Référence != objet** : une variable référence peut être **null** ou ne pas encore désigner d'instance.
- **Java passe tout par valeur**, même les références.
- Ne pas oublier **super(...)** si la superclasse n'a pas le constructeur attendu.
- **private** hérite dans l'état mais n'est pas directement accessible au code de la sous-classe.
- **Map** appartient au framework *Collections* mais n'est pas une sous-interface de **Collection**.
- Fermer fichiers, streams, connexions et statements; préférer **try-with-resources**.
- Une exception **checked** doit être attrapée ou déclarée avec **throws**.
- Les **catch** généraux doivent venir après les **catch** plus spécifiques.
- **run()** n'ouvre pas un nouveau thread : utiliser **start()**.
- **volatile** != atomicité; pour une section critique partagée, utiliser **synchronized** si approprié.
- **accept()** est bloquant : un serveur itératif fait attendre les clients suivants.
- Pour un serveur concurrent, ne pas traiter tous les clients dans le thread qui accepte les connexions.

13. DIFFÉRENCES EXPRESS

A	B
Classe = modèle/type	Objet = instance
Instance = une valeur par objet	static = commun à la classe
Héritage = "est un"	Composition = "a un"
Surcharge = paramètres différents	Redéfinition = même signature héritée
InputStream = octets	Reader = caractères
throw = lance	throws = déclare/propage
start() = nouveau flot	run() direct = appel normal
volatile = visibilité	synchronized = visibilité + verrou/atomicité section